

108 AI Desktop Agent — Piano Installabile Completo

Obiettivo: ****Sostituire Claude Code + ChatGPT Desktop**** con un singolo eseguibile installabile che si configura da solo, si integra in ogni shell, resta in system tray, e si auto-aggiorna.

Stato Attuale (v0.3 — Fase A chiusa)

Feature	Stato	Note
Binary compilato (esbuild/bun)	OK	scripts/build.mjs
108ai --install (copia in PATH)	OK	installer.ts idempotente + menu Repair/Update/Uninstall
CLI one-shot (108ai <domanda>)	OK	Con cache, script-store, smart-chunk
Shell interattiva (108ai shell)	OK	REPL con history
Pipe mode (echo x \ 108ai --pipe)	OK	
Background agent (108ai agent)	OK	WebSocket + triage/job schedulers
System tray	OK	systray2 nativo + menu completo; fallback notifier
OAuth browser login	OK	Apri browser, riceve token
Auto-avvio	OK	autostart.ts Win/Mac/Linux
Re-installabile (idempotente)	OK	Menu interattivo su --install
Auto-update	OK	updater.ts + menu tray "Riavvia per aggiornare"
First-run triage 07:00	OK	first-run.ts
Beta package Windows	OK	pnpm package:beta + install-108ai.ps1
MSI/DMG firmato	Mancante	Fase E (post-feedback)

Nota (Fase B/C già integrate)

- Coding assistant (Fase B): shell.executeStream, git., code., search., web., process.* sono già presenti e registrate (vedi docs/desktop-client-master-plan.md).
- Extensibility & store (Fase C): mcp install generico + store install end-to-end via /ui web (tab Store) e POST /api/store/install.

Visione Target (v1.0)

108ai.exe (singolo file, ~25-40 MB)	
— Doppio-click	→ Installa + avvia agent + apre browser login
— Da shell aperta	→ CLI mode (come Claude Code)
— System tray	→ Icona con menu: Open Shell, Dashboard, Pause, Settings, Quit
— Auto-avvio	→ Si registra in Startup (Win) / LaunchAgent (Mac)
— Auto-update	→ Check versione ogni 6h, download + replace silenzioso
— Re-installazione	→ Ri-cliccare exe → rileva installazione, offre: Repair/Update/Uninstall

Opzioni Architetture

Opzione A: Electron-less (Node.js compilato + native tray)

Pro | Contro

Binario leggero (~25 MB) | Tray icon limitata (solo systray2 o node-systray)
 Nessuna dipendenza runtime | No GUI per settings/chat (solo browser redirect)
 Fast startup (< 1s) | Nessun renderer locale
 Identico a Claude Code

Stack: esbuild bundle → pkg o bun compile per single executable. Tray via systray2 (Go-based, 2MB addon). Menu contestuale nativo.

Installer flow:

```
108ai.exe [primo avvio]
1. Rileva se gia' installato (~/.108ai/bin/108ai.exe)
  - SI → "108 AI gia' installato. [Aggiorna] [Ripara] [Disinstalla] [Annulla]"
  - NO → Procede con installazione
2. Copia se stessa in ~/.108ai/bin/108ai.exe
3. Aggiunge ~/.108ai/bin al PATH utente (setx su Win)
4. Registra auto-avvio:
  - Win: HKCU\Software\Microsoft\Windows\CurrentVersion\Run
  - Mac: ~/Library/LaunchAgents/com.108ai.agent.plist
  - Linux: ~/.config/autostart/108ai.desktop
5. Apre browser per OAuth login
6. Salva token in ~/.108ai/config.json
7. Avvia agent background + tray icon
8. Mostra: "Installazione completata. Usa '108ai' in qualsiasi terminale."
```

Opzione B: Tauri (Rust shell + WebView per UI)**Pro | Contro**

UI nativa per chat/settings | Complessita' build (Rust + Node)
 Tray icon nativa e ricca | Binary piu' pesante (~50-80 MB)
 Webview2 su Win (gia' installato) | Richiede WebView2 runtime
 Installer MSI/NSIS nativo | Tempo di sviluppo 3-4x

Opzione C: Hybrid (Node binary + Webview on-demand)**Pro | Contro**

Binario base leggero come A | Complessita' intermedia
 UI opzionale via webview-node | Dipendenza extra per UI
 Tray nativa + menu completo
 Identico a Claude Code per CLI

Raccomandazione

Opzione A (Electron-less) per v1.0.

Motivazione:

- Claude Code e Cursor funzionano senza Electron per la CLI — il mercato valida questo approccio
- Il target PMI vuole "installa e funziona", non "installa 80MB + runtime"
- La dashboard web e' gia' il centro di controllo — l'agent non ha bisogno di UI propria
- Effort: 2 settimane vs 6+ per Tauri
- L'unica feature che richiede GUI (chat) puo' aprirsi nel browser di default

Opzione B (Tauri) per v2.0 se il feedback dei clienti chiede UI locale.

Piano di Esecuzione Dettagliato

Sprint 1 — Installer Idempotente + Auto-avvio (3 giorni)

File da modificare/creare:

File | Azione

src/installer.ts | NUOVO — Logica installer unificata

src/cli.ts | Refactor handleInstall → usa installer.ts

src/autostart.ts | NUOVO — Registrazione auto-avvio OS-specific

src/index.ts | Aggiungere detection primo-avvio / re-installazione

Flusso installer (idempotente):

```
// src/installer.ts
interface InstallState {
  installed: boolean;
  version: string | null;
  autoStartEnabled: boolean;
  pathConfigured: boolean;
  configExists: boolean;
}

async function detectInstallState(): Promise<InstallState> { ... }

async function install(options: {
  silent?: boolean;    // No interactive prompts
  force?: boolean;     // Sovrascrive anche se stessa versione
  skipAutoStart?: boolean;
  skipPathSetup?: boolean;
}): Promise<void> { ... }

async function repair(): Promise<void> { ... }
async function uninstall(): Promise<void> { ... }
async function upgrade(newBinaryPath: string): Promise<void> { ... }
```

Re-installazione (doppio-click su exe già installato):

```
if (detectInstallState().installed) {
  if (currentVersion > installedVersion) {
    // Auto-upgrade silenzioso
    upgrade(process.execPath);
  } else {
    // Menu: [1] Aggiorna [2] Ripara [3] Disinstalla [4] Annulla
    showInteractiveMenu();
  }
} else {
  install();
}
```

Auto-avvio:

```
// src/autostart.ts
// Windows: Registry Run key
function enableWindowsAutoStart(exePath: string): void {
  execSync(`reg add "HKCU\\Software\\Microsoft\\Windows\\CurrentVersion\\Run" /v "108AI" /t REG_SZ /d "${exePath} agent"`)
}

// Mac: LaunchAgent plist
function enableMacAutoStart(exePath: string): void {
  const plist = `<?xml version="1.0" encoding="UTF-8"?>
<!DOCTYPE plist PUBLIC "-//Apple//DTD PLIST 1.0//EN" "...">
<plist version="1.0"><dict>
  <key>Label</key><string>com.108ai.agent</string>
  <key>ProgramArguments</key><array>
    <string>${exePath}</string><string>agent</string>
  </array>
  <key>RunAtLoad</key><true/>
  <key>KeepAlive</key><true/>
</dict></plist>`;
  writeFileSync(launchAgentPath, plist);
}

// Linux: XDG autostart .desktop file
function enableLinuxAutoStart(exePath: string): void { ... }
```

Sprint 2 — System Tray Nativa (2 giorni)

Dipendenza: systray2 (binding Go, supporta Win/Mac/Linux, ~2MB)

Menu tray:

```
108 AI v0.3.0 — Connesso


---


> Apri Shell (terminale)
> Apri Dashboard (browser)


---


Stato: ● Connesso
Tenant: Demo Azienda S.r.l.
Ultimo uso: 2 min fa


---


> Pausa agente
> Impostazioni (browser)


---


> Esci
```

Icone:

- Verde: connesso + idle
- Giallo: processing / azione in corso
- Rosso: disconnesso
- Grigio: in pausa

File: src/tray.ts — refactor completo per usare systray2 invece di node-notifier.

Sprint 3 — Auto-Update (2 giorni)

Flusso:

Ogni 6 ore:

1. GET /api/desktop-agent/version → { latest: "0.4.0", url: "..." }
2. Se latest > current:
 - a. Download in ~/.108ai/tmp/108ai-new.exe
 - b. Verifica SHA-256 (dal server)
 - c. Notifica tray: "Aggiornamento disponibile. Riavvia per applicare."
 - d. Al prossimo avvio (o click "Riavvia"):
 - Copia new → bin/108ai.exe (Windows: rename trick)
 - Riavvia processo

Windows rename trick:

```
// Non puoi sovrascrivere un .exe in esecuzione su Windows.
// Soluzione: rename vecchio → .old, rename nuovo → .exe, riavvia, delete .old
```

Sprint 4 — Shell Integration Avanzata (3 giorni)

Goal: 108ai da qualsiasi shell funziona come claude — shell mode interattivo con contesto directory, git awareness, file editing.

Funzionalità da aggiungere:

Feature | Stato attuale | Target

REPL interattivo | OK (shell.ts) | Migliorare: multiline, syntax highlight
 Context awareness (git, cwd) | Parziale | Invia sempre branch/status/cwd al gateway
 File editing inline | Mancante | Diff-based edit come Claude Code
 History persistente | Mancante | ~/.108ai/history.jsonl
 Completamento comandi | Mancante | Tab-completion per file/dirs
 Spinner/progress | Mancante | Ora output, streaming token-by-token
 /help, /clear, /model | Mancante | Comandi slash come Claude Code
 Multi-turn context | Parziale | Mantieni conversation_id tra messaggi

Comandi slash da implementare:

```
/help    — Mostra comandi disponibili
/clear   — Pulisci contesto conversazione
/model   — Cambia tier (fast/balanced/powerful)
/files   — Lista file nel contesto
/add <path> — Aggiungi file al contesto
/diff    — Mostra ultime modifiche fatte dall'AI
/undo    — Annulla ultima modifica file
/cost    — Mostra token/costo sessione corrente
/compact — Comprimi il contesto (rimuovi messaggi vecchi)
```

Sprint 5 — Build & Distribution (2 giorni)

Binary size target: < 30 MB

Build pipeline:

```
# 1. Bundle TypeScript → single JS file
esbuild src/index.ts --bundle --platform=node --target=node20 \
--outfile=dist/108ai-bundle.js --external:systray2

# 2. Compile to single executable
# Opzione A: Node SEA (Single Executable Application, Node 20+)
node --experimental-sea-config sea-config.json
npx postject dist/108ai.exe NODE_SEA_BLOB dist/sea-prep.blob --sentinel-fuse NODE_SEA_FUSE_fce680ab2cc467b6e072b8b5df199

# Opzione B: pkg (deprecato ma stabile)
npx pkg dist/108ai-bundle.js -t node20-win-x64 -o dist/108ai.exe

# Opzione C: Bun compile (piu' leggero, experimental)
bun build src/index.ts --compile --outfile dist/108ai
```

Raccomandazione build: Node SEA (nativo, nessuna dipendenza, forward-compatible).

Distribution:

- Download diretto da dashboard: /api/desktop-agent/download/:platform
- Pagina web pubblica con 3 bottoni (Windows/Mac/Linux)
- Checksum SHA-256 per ogni release
- NO installer MSI/DMG — e' un singolo file

Sprint 6 — Polish & UX (2 giorni)

- First-run wizard (terminal-based): benvenuto, login, test connessione
- Error messages user-friendly (no stack traces)
- 108ai doctor — diagnostica problemi (connessione, PATH, config)
- 108ai status — mostra stato agent, ultimo heartbeat, tenant
- 108ai login — forza re-login (se token scaduto)
- 108ai config — apre config in editor

Confronto con Competitor

Feature	108 AI Agent	Claude Code	ChatGPT Desktop	Cursor
Singolo exe, no installer	SI	SI (npm global)	NO (installer)	NO (installer)
Shell integration	SI	SI	NO	Parziale
System tray	SI	NO	SI	NO
Auto-avvio	SI	NO	SI	NO
Auto-update	SI	npm update	SI	SI
Re-installabile	SI (idempotente)	npm reinstall	SI	SI
OAuth login via browser	SI	API key manuale	Account Apple/Google	Account
Pipe support	SI	NO	NO	NO
File editing	SI	SI	NO	SI
Shell execution	SI	SI	NO	SI
Memory persistente	SI (server)	File locale	Limitata	NO
Multi-model (DeepSeek/Qwen)	SI	Solo Claude	Solo GPT	Multi
Costo	Incluso nel piano	\$20/mo	\$20/mo	\$20/mo

Effort & Timeline

Sprint | Effort | Output

1. Installer + Auto-avvio | 3 giorni | Install idempotente, autostart, re-install menu
2. System Tray | 2 giorni | Tray nativa con menu e stati
3. Auto-Update | 2 giorni | Background check + seamless upgrade
4. Shell Avanzata | 3 giorni | Slash commands, history, streaming, context
5. Build & Dist | 2 giorni | Node SEA, CI/CD, download page
6. Polish | 2 giorni | Doctor, status, first-run UX
- TOTALE | 14 giorni | v1.0 release-ready

Rischi & Mitigazioni

Rischio | Impatto | Mitigazione

- systray2 non funziona su tutti i DE Linux | Medio | Graceful fallback (solo CLI, no tray)
- Node SEA non supporta native addons | Alto | Pre-compilare systray2 staticamente o usare subprocess Go
- Windows Defender blocca exe non firmato | Alto | Code signing certificate (\$200-400/anno), oppure istruzioni whitelist
- Auto-update rompe installazione | Critico | Atomic replace + rollback automatico se crash entro 10s
- OAuth flow non funziona dietro proxy aziendale | Medio | Fallback: 108ai login --token <manuale>

Decisione: Build Tool per Binary

Opzione | Size | Startup | Native Addons | Maturita'

- Node SEA | ~30 MB | ~200ms | NO (va in subprocess) | Stabile da Node 21
- pkg | ~45 MB | ~300ms | SI (partial) | Deprecato
- Bun compile | ~15 MB | ~100ms | NO | Experimental
- Deno compile | ~25 MB | ~150ms | NO | Stabile

Raccomandazione: Bun compile per size minima, con fallback a Node SEA se serve stabilita'.

Next Steps

- Validare Opzione A con prototipo tray (systray2 + esbuild bundle)
- Decidere build tool (Bun vs Node SEA)
- Sprint 1 (installer idempotente)
- User testing con 2-3 utenti non-dev dopo Sprint 4